

Automatic Estimation of FM Synthesis Parameters by Convolutional Neural Network

Anonymous Authors

Abstract—FM synthesis is one of the most renowned techniques for sound synthesis, and although it is an old method, it remains widely used today. It is simple, computationally inexpensive, and capable of producing high-quality results. However, the successful use of this type of audio synthesis presents a significant challenge: selecting the correct parameters for the synthesizer. This process is not precise, and there is no fully deterministic procedure to follow. Musicians are usually guided by intuition and an empirical trial-and-error process. To revive this technique and leverage its benefits, the use of CNN models appears to be a very promising option, as they can help musicians achieve convincing results, imitating real instruments in a short time and at a low cost, or even creating totally unpredictable timbres and effects. Therefore, this work explores this approach by creating a pipeline capable not only of training a CNN model to predict the appropriate parameters that enable an FM synthesizer to resynthesize original audio samples (such as acoustic instruments), but also of exploring the synthesizer’s parameter space broadly, allowing it to reproduce any arbitrary desired sound. The proposed approach proved to be efficient for both objectives: the developed model was able to approximate unseen sound samples (of real instruments), and the error metrics converged to satisfactory values even when using an exploratory dataset generation process (a synthetic dataset specifically constructed to explore the FM synthesizer’s capabilities).

I. INTRODUCTION

Musical expression is one of humanity’s earliest experiences and one of the most valued forms of art. This art form has been cultivated since ancient times. The Chinese, three thousand years before Christ, had already developed complex systems of musical theory, and Greek culture not only cultivated its study but also associated it with religion, assigning divine characteristics to protect and honor music itself ([?]).

Although music is a subjective form of human knowledge, the recorded music industry today moves approximately US\$29 billion ([?]) by year, and so, its value ceases to be only subjective.

It is precisely in this context that the artifice of synthesized sound appears in history. It emerged after the first electronic instruments and, due to the periodic nature of sound, the earliest synthesizers were composed of simple analog oscillators that could be manually controlled through operations such as summing and filtering ([?]).

In the 1980s, with the release of the Yamaha DX7 synthesizer, the history of music synthesis took a major turn. The Yamaha DX7 marked the first major success of a revolutionary technique developed by John M. Chowning at Stanford University and later popularized by Yamaha ([?]).

This event represented a new milestone in sound synthesis, not only because of its popularity, but also because it made the reproduction of both realistic instrument simulations and

entirely new kinds of sounds easily accessible to musicians and composers of all levels. In fact, the success of FM synthesis was so great that the technique remains in use to this day ([?]).

Thus, the primary goal of the present work is to guide a synthesizer based on frequency modulation (FM) to reproduce instrument sound samples, while also developing a process that fully explores the expressive capabilities of the FM synthesizer itself, achieving an approximation for the re-synthesis of any input sound.

This approach aligns with one of the key historical advantages of FM synthesizers: their ability to create sounds that cannot be produced by traditional acoustic instruments.

Although FM synthesis is considered an old technique compared to newer approaches such as wavetable synthesis or GAN-based sound generation, it remains widely used within today’s complex musical ecosystem due to its advantages: it is easy to use, well suited for experimenting with new and even unpredictable sounds (such as spatial effects), and easily adaptable to real-time composition.

So, researchers continue to focus on improving the control of traditional FM synthesis parameters (which, when done manually, can be a tedious empirical process). For instance, [?] implemented a neural network to predict FM synthesizer parameters using the frequency-domain sounds representation as input. Similarly, [?] proposed an interesting TCN-based approach, coupling the FM synthesizer directly within the network training process.

The current work, although, wants to employ a Convolutional Neural Network (CNN) to achieve its goals, because CNNs enabling the direct use of raw audio samples as model input while serving as an excellent feature extractor for sound signals ([?]). CNNs are also well known for their ability to economize computational resources when handling high-dimensional data. Therefore, CNNs are a strong candidate for this type of research task.

Furthermore, beyond a simple re-synthesis exercise, the present work aims to develop a process that allows the model to achieve a broad understanding of the entire parameter space of the applied FM synthesiser. The idea behind the proposed methodology is that if the model can learn the FM synthesizer’s parameter space in a comprehensive way, it can then be used to simulate any sound within the synthesizer’s capabilities, making the experience of a hypothetical future user (a musician or composer) much more efficient in achieving any desired sound (allowing good integration into real compositions workflows).

The remainder of this paper is organized as follows. Section ?? presents the fundamental concepts related to music.

Section ?? discusses the related work. Section ?? describes the details of the proposed approach and the experimental setup. Section ?? presents and analyzes the results. Finally, Section ?? provides the conclusions of this work and outlines directions for future research.

II. MUSICAL CONCEPTS

Musical computation is a broad, interdisciplinary field that encompasses the computational processing of music and sound. A comprehensive overview is beyond the scope of this paper; however, for the present study, a basic understanding of key sound properties is necessary, particularly in the context of synthesis and re-synthesis tasks:

Timbre is commonly described as the “color” of sound. From a signal-processing perspective, it is determined by the distribution, relative amplitudes, and temporal evolution of the harmonic components present in an audio signal. In acoustic instruments, these components arise from multiple simultaneous vibration modes, whose behavior depends on physical characteristics such as material, geometry, and excitation mechanism. This complex interaction produces a distinctive spectral structure that characterizes each instrument’s timbre. To analyze this property, audio signals are commonly transformed from the time domain to the frequency domain using Fourier-based techniques [?].

A. Audio digitization

To digitize an audio signal, a process known as sampling is used [?]. For human hearing, a sampling frequency of 20 kHz is generally sufficient. However, when focusing on musical instruments, a sampling rate of 16 kHz is often adequate. Therefore, to ensure compatibility with the NSynth dataset (presented below), this work adopts an audio sampling rate of 16 kHz.

B. Frequency Modulation Sound Synthesis

One of the most popular sound synthesis techniques is Frequency Modulation Synthesis (FM Synthesis), which was introduced by John M. Chowning [?], which explored the effects of frequency modulation when applied with high frequencies, close to the audible range, instead of the traditional use of inaudible frequencies.

In summary, the FM synthesis technique allows the addition of multiple sidebands around the base frequency for each modulator used in the synthesizer algorithm. As a result, rich spectral audio can be created using only a few modulators, and a sound designer can, through an empirical process, create arbitrary sounds by adjusting the synthesizer parameters ([?]).

This type of synthesis was popularized by the Yamaha DX7 synthesizer and remains in use today due to its efficiency, low computational cost, and flexibility ([?]).

C. Audio comparison

The task of audio comparison is not straightforward, especially when the goal is to achieve perceptual similarity from a human hearing perspective. It can be complex, computationally demanding, and inherently subjective; however, several

well-established and efficient methods can be employed to address this challenge.

According to [?], three metrics are commonly used for this purpose. The **FFT distance** is computed by applying the Fast Fourier Transform (FFT) to the two audio samples and measuring the Euclidean distance between their spectra, typically normalized to the $[0, 1]$ range. The **Short-Time Fourier Transform (STFT) distance** follows the same principle but is calculated over short, overlapping time windows, providing time-localized spectral information. Finally, the **Log-Mel spectrogram distance** is based on the Log-Mel representation, which adapts the FFT spectrum to better reflect human auditory perception by mapping linear frequencies onto the Mel scale [?], [?].

In addition to these metrics, [?] also recommends the use of the Euclidean distance between amplitude envelopes. While this approach is valuable for evaluating temporal dynamics, the present work focuses specifically on timbral similarity; therefore, envelope-based comparison is considered outside the scope of this study.

III. RELATED WORK

Several studies have explored the use of machine learning to estimate parameters for sound synthesizers. The main contribution of the present work lies in the use of convolutional neural networks (CNNs) operating directly on raw audio signals. CNNs are particularly well-suited for this task due to their ability to process high-dimensional inputs, capture temporal structures in audio signals, and provide near-real-time inference after training.

[?] proposed the re-synthesis of instrumental sounds using machine learning models combined with an FM synthesizer. In that work, however, CNNs and raw audio were not used; instead, the input consisted of STFT spectrograms, a frequency-domain representation that may simplify learning but does not fully exploit temporal information present in the signal.

[?] introduced DDX7, a differentiable FM synthesizer designed for the re-synthesis of musical instrument sounds. Their approach employs a Temporal Convolutional Network (TCN) and integrates the Mel-spectrum directly into the loss function, aligning optimization with perceptually motivated audio metrics. While effective, this method requires embedding the synthesizer within the training loop, resulting in higher computational and memory costs. In contrast, the present work emphasizes generalization through large-scale randomized exploration of the FM parameter space, without incorporating the synthesizer into the optimization process.

Finally, [?] presented a comprehensive survey of one-dimensional CNNs and their applications, highlighting their effectiveness for sequential data such as audio signals. This work supports the suitability of CNN-based architectures for learning temporal patterns directly from raw audio.

IV. METHODOLOGY

This section introduces our proposed approach of using AI to guide an FM synthesizer in simulating any instrument or any

desired sound. More specifically, we propose a Convolutional Neural Network (CNN) to analyze a sound signal sample and predict the correct parameters for an FM synthesizer to re-synthesize the “same” timbre, enabling the synthesizer to simulate the desired “voice” playing any song. Since this idea is still quite abstract and broad, we consider the following steps:

- 1) **Implementing FM synthesizer:** Implement a simple FM synthesizer, which can later be used both to generate the dataset and to test the ability to re-synthesize a sound.
- 2) **Generating the dataset:** Generate an synthetic audio dataset composed of random samples, each created by randomly selecting FM synthesizer parameters. This essentially records tuples of [audio sample; parameter values] (where the parameters are the target values), and explores the parameter space on a large scale.
- 3) **Training the CNN:** Train a CNN to predict the correct parameters for the same FM synthesizer, given only an audio sample.
- 4) **Evaluating the Model:** Evaluate the model’s results using the NSynth dataset to test its generalization capabilities.

Frequency Modulation Synthesis was chosen because it is one of the most common and effective methods for generating audio samples, owing to its simplicity and low computational cost. However, the effective use of this technique depends on advanced knowledge to select the correct parameters for generating good results.

Therefore, the ability to simplify parameter selection constitutes a strong justification for this study, and AI models appear as an affordable way to learn the right patterns for controlling an FM synthesizer. The basic idea is that a model can be produced to guide an FM synthesizer in mimicking any sound, thereby optimizing the effort to compose electronic music.

Furthermore, in terms of computational cost, using an AI model to predict parameters and guide an FM synthesizer is much more effective than applying a Generative Neural Network to generate an audio. Additionally, a synthesizer is particularly well-suited for real-time composition and electronic musician performance.

Regarding the configuration aspects, all experiments were conducted in an environment composed of Python (3.11.2), TensorFlow (2.19.0), Keras (3.11.1), Nvidia CUDA (12.5.82), Nvidia cuDNN (3.0.75), Nvidia driver (535.247.01), and CUDA (12.2), using an NVIDIA GeForce GTX 1650 GPU with 4GB of memory.

A. Implementing FM synthesizer

There are several well-known FM synthesizers on the market, but most are not open-source. Additionally, commercial synthesizers often have a vast number of parameters that can be used to control the audio production process. For example, Native Instruments’ FM8 can use around 1,000 parameters ([?]), and Teenage Engineering’s OP-1 synthesizer can be configured with up to 10^{76} parameters ([?]).

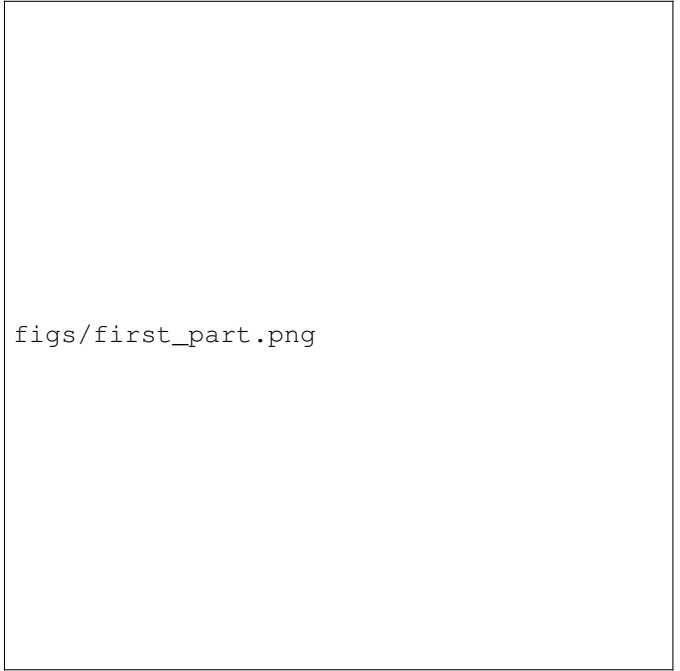


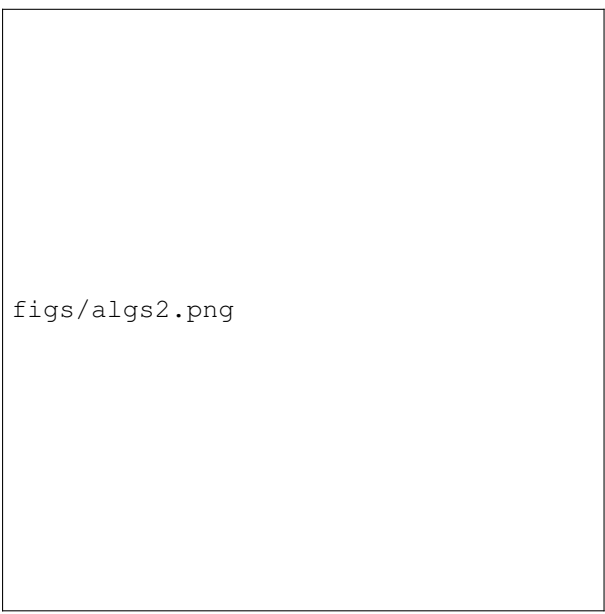
Fig. 1: Synthesis implementation and dataset generation.

This incredibly large number of parameters is intended to give users fine-grained control over the generated audio, allowing for a wide variety of pitches (simulating a broad range of instruments and even non-instrument sounds, such as bird songs or car horns, among others). However, since the scope of this project is not focused on a commercial solution but rather on research purposes, a simple FM synthesizer will suffice.

The implemented FM synthesizer has only 6 operators, inspired by the classic Yamaha DX7, which was one of the most disruptive milestones in the history of electronic music composition. In fact, this simple setup is enough to cover all FM synthesis theories and achieve good performance during AI model evaluation. In summary, the implemented FM Synthesizer (Figure ??) consists of 5 modulator operators, 1 carrier operator, 1 ADSR envelope, and 2 synthesis algorithms (one completely serial, and another with 3 serial operators followed by 2 parallel operators).

To summarize, the implemented synthesizer renders the audio signal according to the following steps:

- 1) Generate each operator signal by applying it to modulate the phase of the next operator in the sequence. In this implementation, only the phase is varied (similar to the Yamaha DX7), whereas a full FM synthesis would also be able to vary the frequency. This simplification helps to avoid complex noise-handling issues and promotes numerical stability. All signals are rendered at a 48 kHz sample rate.
- 2) Combine the output of each operator according to the synthesis algorithm (when there are parallel operators, they are summed).



figs/algs2.png

Fig. 2: FM Synthesizer Algorithms.

- 3) Apply the ADSR envelope to the signal.
- 4) Downsample the signal from 48 kHz to 16 kHz (applying also a low-pass filter to avoid aliasing).

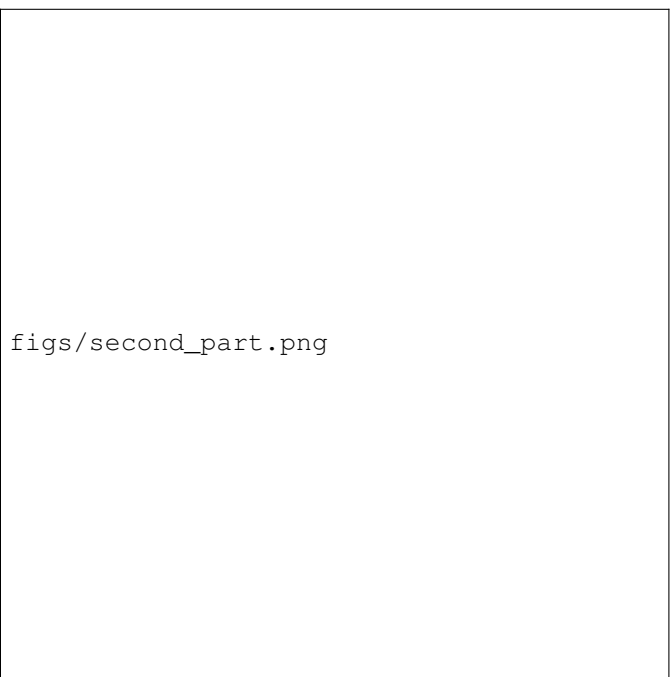
It is important to highlight that the high sample rate of 48 kHz was chosen to allow the FM synthesis to work well enough, since at high frequencies, there is a risk of obtaining aliasing and noise (due to the extremely high frequency components generated by the normal spacing of the FM operators). Furthermore, the low-pass filtering, used to downsample, is also useful to prevent the aliasing problem (after the downsampling process).

Finally, it is important to highlight that the entire synthesizer relies on 16 input parameters, consisting of the beta and ratio values for each modulator and for the carrier (totaling 12 parameters), plus 4 parameters for the ADSR envelope (attack, decay, sustain, and release times). As mentioned earlier, it is a very simple FM synthesizer, yet fully featured and sufficient for the purposes of the current research.

B. Generating the dataset

This work utilizes two datasets, each serving a distinct purpose: the generated dataset, referred to below as the synthetic dataset, which is used for training; and the NSynth dataset, which is used to evaluate the model's generalization capabilities. However, in this section, only the first dataset will be considered, while the second will be addressed in Section ??.

One of the most challenging steps in the KDD process is obtaining and processing data. However, for the purposes of this work, an effective approach can be used. Similar to Claesson's approach [?], since the goal is to achieve good control over the synthesizer parameters, it is sufficient to generate many samples of synthesizer parameters and their respective audio outputs.



figs/second_part.png

Fig. 3: Synthesis implementation and dataset generation.

The dataset used to train the model will consist of 5,000 samples generated by the implemented FM synthesizer, each with a duration of 4 seconds, monophonic audio (single-channel), a 16 kHz sample rate (with samples originally generated at 48 kHz and subsequently downsampled to ensure compatibility with the NSynth dataset), pitches varying between 20 Hz and 6,000 Hz, and randomly generated ADSR envelopes.

The expectation is to achieve a level of learning where the model can accurately predict the parameters for a wide range of sounds that can be generated by the implemented FM synthesizer.

However, as the main purpose is to re-synthesize real instrument samples, the randomness of this dataset should be controlled to achieve reasonable music property samples, or, in other words, the dataset was constructed respecting the well-known limits and characteristics of real instruments:

- The minimum frequency for the carrier is 20 Hz, and the maximum is 6 kHz (for example, the maximum frequency for a violin is approximately 3 kHz; another important aspect for this maximum choice is that we need to stay under 8 kHz due to the output sample rate, because of the Nyquist frequency). The frequency is chosen by a uniform random distribution.
- The minimum ratio for each modulator is 1/8, and the maximum is 8 (to avoid aliasing in the final signal). Two methods are used to select the ratio: a) with 90% probability, the ratio will be a random uniform choice between discretized values (for example: 1/8, 1/6, ..., 2, 3, 54, ..., 8; the idea is to simulate a real instrument); and b) with 10% probability, the ratio will be a uniform

log distribution between 1/8 and 8 (allowing random metallic sounds, like bells, but respecting human auditory perception, where the difference between low frequencies is more perceived than between high frequencies).

- The minimum beta is 0, and the maximum is 8 (also to avoid aliasing in the final signal). The beta is chosen by a uniform random distribution, controlled such that 20% of the values are between 0 and 0.5, 60% are between 0.5 and 3, and 20% are between 3 and 8 (prioritizing rich timbres and avoiding pure sinusoidal sounds, as well as excessive metallic sounds).
- The amplitude is fixed at 1.0, to avoid different combinations of beta and amplitude generating the same frequency spectrum and the same timbre (prioritizing the CNN learning).

Regarding the ADSR envelope, the attack, decay, sustain, and release parameters were randomly sampled from uniform distributions within predefined intervals. Specifically, the attack time ranged from 0.005 to 0.5 s, the decay time from 0.02 to 0.6 s, the sustain level from 0.1 to 0.9, and the release time from 0.05 to 0.8 s.

With these additional adjustments, the synthetic dataset will be more representative of simulating real instruments.

As each sample in the generated dataset is a complete 4-second sound (containing 64,000 audio samples, which corresponds to 16 kHz multiplied by 4 seconds), so a simple 75/25 train-test split was used, resulting in 3,750 audio files for training and 1,250 for testing.

But, from these 3,750 audio files, 20% of them will be used for validation, resulting in a total of 3,000 samples for training and 750 samples for testing.

C. Training the CNN

As described in Section ??, the main characteristic of the sound that should be reproduced to achieve the main work goal is the “timbre“, which is defined as the combination of frequencies present in the sound. It is clear that techniques such as filtering and Fourier Transform can be used to extract timbre components from an audio signal; however, they are insufficient for controlling an FM Synthesizer. In fact, the FM Synthesis control is, normally, an experimental process involving trial and error.

Additionally, each FM Synthesis operator generates additional frequency components in the signal, and there is no practical method to predict the correct parameter settings. However, as a mathematical function, it is easy to see that this synthesis pattern can be easily approximated using AI models.

Thus, the current work aims to use a CNN model, since it is a very effective method for learning patterns, respecting the internal data structure, and without requiring extensive pre-processing work. In other words, a CNN can simultaneously learn the time-dependent aspects of an audio signal and can act as a pattern extractor by applying a sequence of filtering operations, allowing the use of the raw audio format as input.

To train the CNN model, the already presented synthetic dataset will be used. Its main advantage is that it contains both the audio signal and the corresponding parameters used to generate each sample. In addition, this approach covers a wide range of the parameter space.

Consequently, the training process is straightforward and is expected to yield good performance.

Figure ?? presents the overall scheme of this training process.

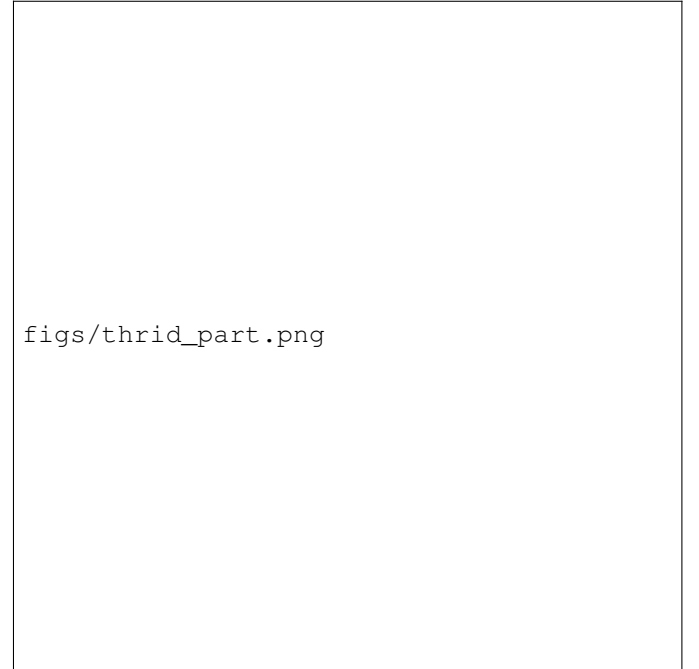


Fig. 4: Model training and optimization.

The model architecture used is as follows, where the first covers the feature extraction (CNN), and the second covers the regression (a common MLP):

The complete model is simply the sum of the two parts. But it is important to highlight the hyperparameters of each.

The feature-extraction part (CNN) comprises three sequential Conv1D–MaxPooling blocks. The first Conv1D layer uses 8 filters with a kernel size of 4 and stride 1, followed by a MaxPooling1D layer with pool size 16. The second Conv1D layer uses 16 filters with a kernel size of 8 and stride 1, followed by MaxPooling1D with pool size 16. The third Conv1D layer uses 32 filters with a kernel size of 16 and stride 1, followed by a final MaxPooling1D layer with pool size 16.

The regressor network part consists of two fully connected hidden layers followed by an output layer. The first dense layer uses the ReLU activation function and is followed by a dropout layer with a rate of 0.5 to mitigate overfitting. The second dense layer also employs the ReLU activation function. The output layer uses a linear activation (i.e., no activation function) to perform parameter regression. Model optimization is carried out using the Nadam optimizer.



Fig. 5: Overview of the entire Model Architecture

Additional training settings include normalization of the target variables to zero mean and unit variance prior to training. The Nadam optimizer was employed, with mean squared error (MSE) used as the loss function, while mean absolute error (MAE) and MSE were recorded as evaluation metrics. The model was trained for up to 200 epochs using a batch size of 32, with early stopping enabled (patience of 20 epochs) and restoration of the best-performing weights.

1) *Hyperparameters Optimization*: To obtain the final architecture, a random search was performed over activation functions, regularization strategies, optimizers, and dropout rates. The best configuration was selected based on validation set performance. Although conceptually similar to a traditional grid search, this approach evaluates random combinations of hyperparameters, enabling broader coverage of the search space with reduced computational cost. The MLflow tool was used to track and compare training runs (additional details are available in the project repository).

The selected configuration employs the ReLU activation function, bias terms enabled in both the CNN and the regressor, a dropout rate of 0.5 in the regressor, no kernel regularization in the CNN, and L2 kernel regularization in the regressor.

D. Evaluating the Model

After training the CNN, a new dataset is used to evaluate the model's ability to re-synthesize real instruments by calculating specific metrics focused on the FM synthesizer's capacity to reproduce the same timbres.

The NSynth dataset, part of Google's Magenta project, contains numerous real instrument samples, including both acoustic and electronic instruments, as well as some synthetic sounds generated by professional synthesizers. This dataset

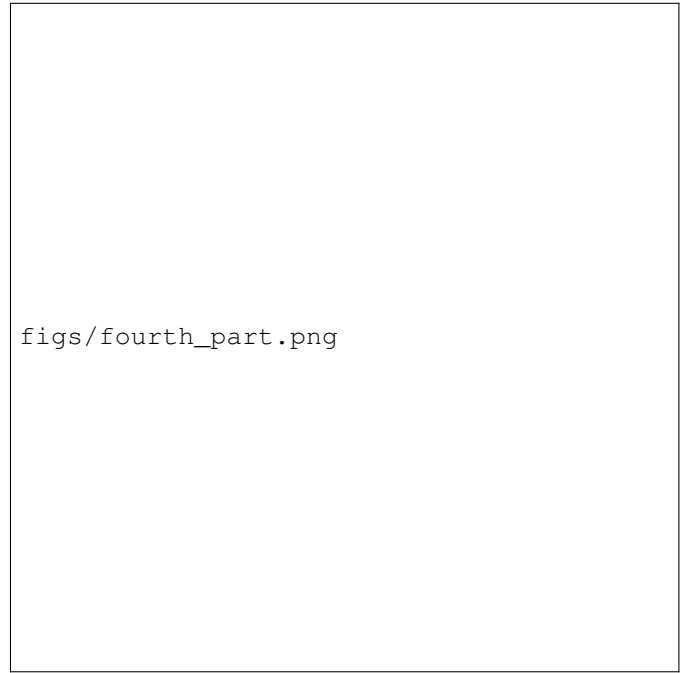


Fig. 6: Results evaluation.

has a configuration similar to that of the generated one used for training, and its main purpose is to evaluate the model's generalization capabilities, since the model has never encountered it before.

Only the test split of the NSynth dataset was used in the evaluation, comprising 4,096 monophonic audio samples, each with a duration of 4 seconds and a sampling rate of 16 kHz. The dataset covers 11 instrument families, with pitches ranging from 27.5 Hz to 4,186.01 Hz, and employs amplitude envelopes consisting of up to 3 seconds of sustain followed by a 1-second decay.

The basic idea is to use the NSynth test samples as input to the model and, after predicting the synthesizer parameters, employ the implemented synthesizer to re-synthesize all samples. Finally, audio comparison metrics are calculated between each input and its corresponding generated sound, based on the proposed evaluation measures:

- FFT distance: Used to compare the frequency components present in an audio signal.
- STFT distance: Used to compare the variation of frequency components over time.
- Log-mel-spectrogram distance: Used for a similar purpose as FFT distance, but typically models human cognitive pitch perception more accurately.

However, before using the predicted parameters as input to the synthesizer, a simple function was applied to approximate each ratio parameter to the nearest discrete ratio, according to the common behavior of musical harmonics (the same set of ratios used to randomize 90% of the generated dataset). Furthermore, some clipping operations were also applied to enforce the same limits imposed on the randomly generated

dataset, such as maximum and minimum values for beta parameters.

In order to achieve a good evaluation, the proposed metrics were also used to compare each NSynth audio with a baseline, which consists of a pure sinusoidal signal, with a frequency of 440 Hz (i.e., the A4 note, which is used as a reference to tune the majority of instruments).

V. EXPERIMENTAL RESULTS AND DISCUSSION

This section presents the final results of the model, considering both the training metrics and the audio comparison metrics. The former is used to evaluate the model’s ability to learn the entire parameter space, while the latter assesses its generalization capabilities, particularly its ability to emulate real instruments.

A. Training Results

The training and validation curves are shown in Figure ?? . The model exhibits stable convergence, with no evidence of overfitting throughout the training process.

figs/mae_mse_evolution.png

Fig. 7: Evolution of MAE and MSE during training.

After training, the model was evaluated on a held-out portion of the synthetic dataset. The predicted parameters were transformed back to their original scale, yielding a mean absolute error (MAE) of 6.488, a mean squared error (MSE) of 1410.097, and a root mean squared error (RMSE) of 37.551. These results indicate that the model is able to accurately reproduce the synthesizer parameters generated during the synthetic data creation process.

B. Audio Comparison Results

As presented in Section ??, after training, the model’s generalization capabilities were tested through a process of

re-synthesizing the NSynth dataset. The performance was evaluated by comparing the audio distance metrics between the original samples and the baseline samples with those between the re-synthesized samples and the original ones:

TABLE I: Comparison between Resynthesis and Baseline with Improvement % (considering mean and standard deviation). [como você calculou o improvement??]

	FFT	STFT	Log Mel (raw)	Log Mel (normalized)
Resynthesis	12649.9 (6208.1)	2366.4 (1132.2)	138.73 (47.013)	0.0086 (0.0029)
Baseline	33508.8 (1887.36)	7290.4 (434.87)	184.87 (30.136)	0.0115 (0.0019)
Tuning (%)	62.270	67.540	24.960	24.930

Note that the Log Mel metric was compared, including a normalized version, which is obtained by dividing the raw Log Mel value by the product of the number of Mel bands and the time window quantity.

Furthermore, to facilitate a better understanding, the metrics were also compared by grouping the dataset by instrument family. This allows for the evaluation of the model’s performance on different classes of sounds. Tables ?? and ?? show the average and standard deviation values, respectively. We can note that the top three best results were obtained for keyboard, guitar, and mallet families.

TABLE II: Average values by Instrument Group.

Group	FFT	STFT	Log Mel (raw)	Log Mel (normalized)
bass	11232.7	2144.32	135.610	0.00841
brass	14371.1	2682.95	167.617	0.01039
flute	14611.3	2872.69	141.819	0.00879
guitar	9692.0	1867.47	119.853	0.00743
keyboard	9619.3	1772.92	119.529	0.00741
mallet	8085.8	1632.45	120.610	0.00748
organ	18360.1	3453.13	166.433	0.01032
reed	12740.0	2267.08	140.701	0.00872
string	12883.7	2390.65	139.365	0.00864
vocal	31028.9	5271.26	212.646	0.01319

VI. CONCLUSION

This work aimed to develop and evaluate a technique capable of achieving two primary objectives: re-synthesizing the sounds of real musical instruments and exploring the synthesizer’s parameter space in a randomized manner to assess the model’s generalization capacity. The proposed convolutional neural network (CNN) model demonstrated strong performance in both aspects, effectively reproducing real instrument timbres and showing satisfactory generalization when exposed to unseen parameter configurations. The re-synthesized audios, when compared to the original samples from the NSynth dataset, confirmed the model’s ability to approximate realistic timbral characteristics, while maintaining consistent performance across different instrument families. The stability of

the normalized log-mel metric across all categories suggests that the model is not overly biased toward specific timbral structures, reinforcing its robustness and flexibility. These results validate the initial hypothesis that CNNs are highly efficient in extracting relevant spectral and temporal features from audio signals, and that randomized dataset generation can effectively guide the learning process, enabling a level of control over FM synthesis parameters sufficient to emulate real-world sounds.

Despite these promising outcomes, the approach presents some limitations that delineate directions for future research. First, the use of a dataset restricted to musical instruments limits the generalization potential of the system; therefore, future experiments should incorporate more diverse sound sources, such as environmental or synthetic noises, to enhance creative applicability. Additionally, the adoption of a single linear ADSR envelope constrained the diversity of synthesized dynamics, indicating that broader variations should be implemented to better model non-instrumental or abstract

sound categories. Another limitation concerns the exploration of FM synthesis algorithms, as only two were tested, and a single configuration yielded satisfactory results; expanding this investigation to include more complex operator topologies could significantly improve the synthesis flexibility.

Future work should thus focus on testing alternative neural architectures, such as autoencoders, to improve feature representation and generalization, as well as combining CNN-based feature extraction with other strategies found in the literature, including the use of STFT-based regression inputs or perceptually informed loss functions such as the log-mel metric. Moreover, the application of AutoML techniques could optimize the regression stage of the pipeline, potentially enhancing the model's predictive performance. Finally, retraining the model using a professional FM synthesizer implementation would provide a more realistic framework for evaluation and deployment, advancing the state of neural sound synthesis and broadening its relevance for both academic research and creative audio production.